

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: JOHN DIEGO MONTAÑO ANGULO

PARALELO: A

SEMESTRE: QUINTO

ACTIVIDAD: 1 Sistema de Gestión de Incidentes Help Desk

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad _____

Actividad # _____

(Desarrollo de la actividad propuesta)

Actividad 1 – Análisis del Problema

Sistema de Gestión de Incidentes Help Desk

Según la guía de la actividad, la primera parte consiste en realizar el análisis del problema identificando actores y requerimientos del sistema.

1. Introducción

En la actualidad, las organizaciones dependen constantemente de los sistemas informáticos para desarrollar sus actividades diarias. Debido a esto, es necesario contar con herramientas que permitan gestionar de manera organizada los incidentes técnicos relacionados con hardware, software y redes.

El presente trabajo tiene como finalidad diseñar la arquitectura lógica de un sistema Help Desk utilizando diagramas UML, patrones de diseño y herramientas de control de versiones. La propuesta busca mejorar la administración de incidencias técnicas dentro de un entorno profesional, permitiendo optimizar el seguimiento, asignación y resolución de tickets.

2. Análisis del Problema

En muchas instituciones y empresas, los incidentes técnicos son gestionados manualmente, provocando dificultades para controlar el seguimiento de los casos reportados. Entre los principales problemas encontrados se identifican:

- Pérdida de información de incidencias.
- Retrasos en la atención técnica.
- Falta de control en la asignación de responsables.
- Dificultad para realizar seguimiento de tickets.
- Problemas para mantener historiales organizados.



Para solucionar esta problemática, se propone implementar un sistema Help Desk capaz de automatizar el registro y administración de incidencias técnicas mediante una plataforma organizada y escalable.

2. Objetivo General

Diseñar la arquitectura lógica de un sistema Help Desk aplicando diagramas UML, patrones de diseño y herramientas de control de versiones para mejorar la gestión de incidencias técnicas.

3. Actores del Sistema

3.1 Usuario Final

Es la persona encargada de reportar incidentes dentro del sistema.

Funciones:

- Registrar tickets.
- Consultar estados de incidencias.
- Recibir información de actualización.
- Adjuntar detalles del problema.

3.2 Técnico de Soporte

Responsable de analizar y resolver los incidentes asignados.

Funciones:

- Revisar tickets.
- Actualizar estados.
- Registrar soluciones.
- Dar seguimiento a incidencias.



3.3 Administrador

Encargado de supervisar el funcionamiento general del sistema.

Funciones:

- Gestionar usuarios.
- Asignar técnicos.
- Administrar categorías.
- Supervisar reportes.

4. Requerimientos Funcionales

Los requerimientos funcionales representan las acciones principales que deberá realizar el sistema.

RF1 – Registro de Usuarios

El sistema deberá permitir registrar usuarios según sus roles.

RF2 – Inicio de Sesión

Los usuarios podrán acceder mediante correo electrónico y contraseña.

RF3 – Creación de Tickets

El sistema permitirá registrar incidencias técnicas especificando descripción, categoría y prioridad.

RF4 – Asignación de Técnicos

El administrador podrá asignar tickets a técnicos especializados.

RF5 – Gestión de Estados

Los tickets podrán cambiar de estado:

- Abierto
- En proceso



- Resuelto
- Cerrado

RF6 – Historial de Incidencias

El sistema almacenará el historial de cambios y soluciones aplicadas.

5. Requerimientos No Funcionales

RNF1 – Seguridad

El sistema deberá proteger el acceso mediante autenticación.

RNF2 – Escalabilidad

La plataforma deberá soportar crecimiento de usuarios e incidencias.

RNF3 – Disponibilidad

El sistema deberá permanecer operativo la mayor parte del tiempo.

RNF4 – Rendimiento

Las consultas y búsquedas deberán ejecutarse rápidamente.

RNF5 – Mantenibilidad

La arquitectura deberá permitir futuras mejoras y modificaciones.

6. Diagrama de Casos de Uso

En esta sección se presenta el diagrama de casos de uso del sistema Help Desk.

7. Diagrama de Clases UML

El diagrama de clases representa la estructura principal del sistema y las relaciones entre sus componentes.

8. Diagrama de Secuencia



El diagrama de secuencia representa el flujo de interacción entre los módulos principales del sistema durante el proceso de creación y atención de un ticket.

Participantes:

- Usuario
- Interfaz
- GestorTickets
- Técnico
- Sistema de Notificaciones

Flujo del proceso:

1. El usuario registra un ticket.
2. El sistema almacena la incidencia.
3. El administrador asigna un técnico.
4. El técnico recibe la notificación.
5. El técnico actualiza el estado del ticket.
6. El usuario recibe la confirmación.

9. Aplicación de Patrones de Diseño

Para fortalecer la arquitectura del sistema se aplicaron distintos patrones de diseño orientados a mejorar la organización y reutilización del software.

9.1 Patrón Singleton

El patrón Singleton se aplicó al gestor principal de tickets para garantizar que exista una única instancia encargada de administrar las incidencias del sistema.

Beneficios:

- Centralización del control.
- Evita duplicación de gestores.
- Mejora la administración de recursos.



9.2 Patrón Observer

El patrón Observer se implementó para el sistema de notificaciones automáticas. Cuando un ticket cambia de estado, los usuarios involucrados reciben información actualizada.

Beneficios:

- Comunicación automática.
- Mejor seguimiento de incidencias.
- Actualizaciones en tiempo real.

9.3 Patrón Factory Method

El patrón Factory Method permite crear distintos tipos de tickets sin modificar la lógica principal del sistema.

Tipos de incidencias:

- Hardware
- Software
- Redes

Beneficios:

- Flexibilidad.
- Escalabilidad.
- Facilidad para agregar nuevos tipos.

10. Uso de GitHub y Gitflow

Para el control de versiones se utilizó GitHub como plataforma de almacenamiento y Gitflow como metodología de organización.



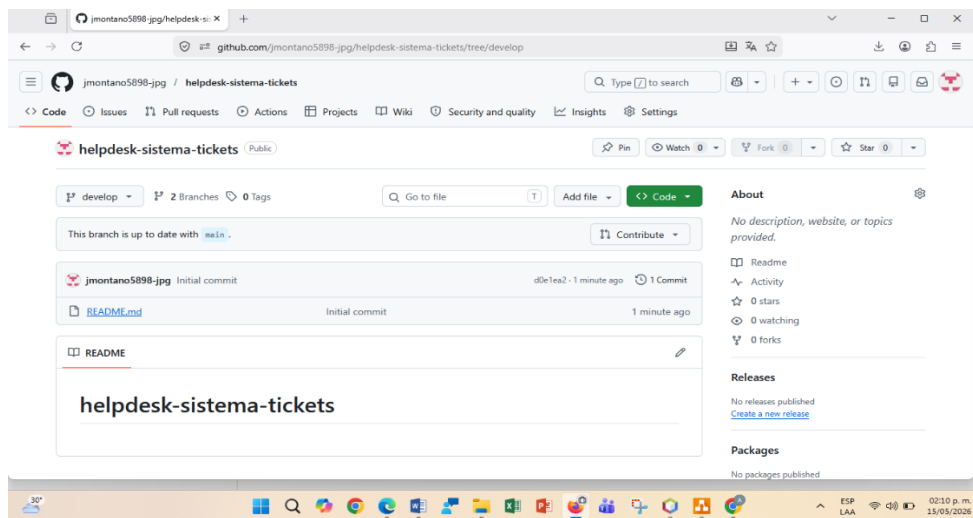
Ramas utilizadas:

- main
- develop
- feature-diagramas

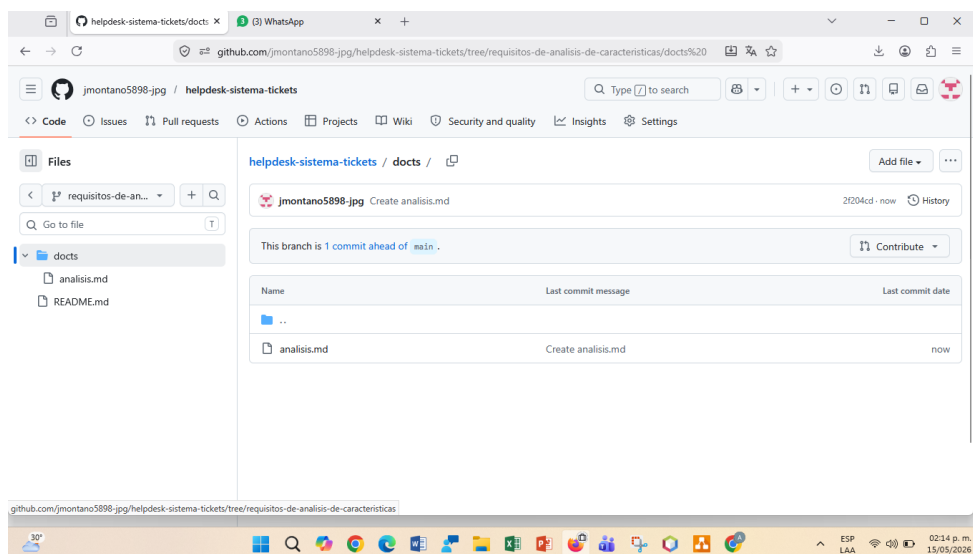
Gitflow permite mantener organizado el desarrollo del proyecto, separando las versiones estables de las funcionalidades en proceso.

11. Evidencias del Proyecto

Evidencia 1 – Repositorio GitHub



(Aquí insertar captura del repositorio)





Parent directory contains leading or trailing whitespace.

helpdesk-sistema-tickets / docts

/ analisis.md in requisitos-de-analisis-de-caracteristicas

Cancel changes Commit changes...

Edit Preview

Spaces 2 No wrap

```
1 # Análisis del Escenario Técnico
2
3 ## 1. Actores del Sistema
4 - *Usuario Final*: Reporta incidentes en el Data Center.
5 - *Técnico de Soporte*: Atiende y resuelve los tickets.
6
7 ## 2. Requerimientos Funcionales
8 - RF1: El sistema debe permitir crear tickets de incidentes.
9 - RF2: El sistema debe notificar al técnico cuando se cree un ticket.
10
11 ## 3. Requerimientos No Funcionales
12 - RNF1: El sistema debe responder en menos de 3 segundos.
13
14
15 ## 4. Guarda el archivo
16 1. Baja hasta donde dice "Commit new file".
17 2. En "Commit message" escribe: Se agrega análisis del escenario técnico
18 3. Deja marcado: "Commit directly to the requisitos-de-analisis-de-caracteristicas branch"
19 4. Haz clic en "Commit new file".]
```

Use `Control + Shift + n` to toggle the `tab` key moving focus. Alternatively, use `esc` then `tab` to move to the next interactive element on the page.

Attach files by dragging & dropping, selecting or pasting them.

helpdesk-sistema-tickets/docts

github.com/jmontano5898-jpg/helpdesk-sistema-tickets/tree/requisitos-de-analisis-de-caracteristicas/docts%20

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Files

requisitos-de-an...

Go to file

docts

analisis.md

README.md

helpdesk-sistema-tickets / docts

Add file

jmontano5898-jpg Create analisis.md 2f204cd · now History

This branch is 1 commit ahead of main.

Contribute

Name	Last commit message	Last commit date
..		
analisis.md	Create analisis.md	now



avinces1033-rgb/proyecto-anal... Informe en Word

github.com/avinces1033-rgb/proyecto-analisis-sistemas/tree/requisitos-de-analisis-de-caracteristicas

proyecto-analisis-sistemas Public

requisitos-de-analisis-de-caracteristicas had recent pushes 1 second ago Compare & pull request

requisitos-de-analisis-de-caract... 3 Branches 0 Tags Go to file Add file Code

This branch is 5 commits ahead of main. Contribute

avinces1033-rgb Add files via upload aeb1c9c · 1 minute ago 6 Commits

docs	Add files via upload	3 minutes ago
Casos.png	Add files via upload	1 minute ago
Clases.png	Add files via upload	3 minutes ago
Diagrama sin titulo.drawio.png	Add files via upload	2 minutes ago
README.md	Initial commit	1 hour ago

README

About

No description, website, or topics provided.

Readme Activity 0 stars 0 watching 0 forks

Releases

No releases published Create a new release

Packages

No packages published Publish your first package

Contributors 1

avinces1033-rgb

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help Sistema-gestor-eventos - Apache NetBeans IDE 23

Projects

- Roxana_Mendoza_practica2_1sc [master]
- Sistema_Garage_UTM [master]
- SISTEMAGARAGEUTM [master]
- SistemaGarajeUTM [master]
- Source Packages
 - sistemagarsjeutm
 - ArbolABR.java [-/A]
 - Cola.java [-/A]
 - ListaDoble.java [-/A]
 - Nodo.java [-/A]
 - NodoArbol.java [-/A]
 - Principal.java [-/A]
 - Vehiculo.java [-/A]
 - Test Packages
 - Libraries
 - Test Libraries
- Sistema-gestor-eventos [master]
- Source Packages
 - sistema.gestor.eventos
 - Asistente.java [-/A]
 - Evento.java [-/A]
 - EventoConcierto.java [-/A]
 - EventoFactory.java [-/A]
 - GestorEventos.java [-/A]
 - IObservadorEvento.java [-/A]
 - Libraries
- SoloImpares [master]
- SoloNumerosImpares [master]
- TABLA_MULTIPLICAR [master]
- TALLER_GUI_SWING [master]
- TAREA_CINE [master]
- atarea001 [master]

Source

```
12 public void avisarSuscriptores(String mensaje) {
13     for (IObservadorEvento p : suscriptores) {
14         p.recibirNotificacion(mensaje);
15     }
16 }
17
18 public void crearEvento(String tipo) {
19     System.out.println("\n=== CREANDO EVENTO ===");
20
21     Evento evento = EventoFactory.generarEvento(tipo);
22
23     if (evento != null) {
24         evento.describir();
25         avisarSuscriptores("Nuevo evento disponible: " + tipo);
26     } else {
27         System.out.println("Error: Tipo de evento no existe.");
28     }
29 }
30
31 public static void main(String[] args) {
```

Output

NetBeansProjects - C:\Users\Usuario\Documents\NetBeansProjects x Sistema-gestor-eventos (run) x

```
=== CREANDO EVENTO ===
Evento: Concierto de musica en vivo programado.
Asistente Maria: Nuevo evento disponible: Concierto
Asistente Luis: Nuevo evento disponible: Concierto
BUILD SUCCESSFUL (total time: 0 seconds)
```

22:59 INS



avinces1033-rgb/proyecto-anal x Informe en Word x +

github.com/avinces1033-rgb/proyecto-analisis-sistemas/tree/requisitos-de-analisis-de-caracteristicas

This branch is 6 commits ahead of main .

avinces1033-rgb Add files via upload 396c0f4 · now 7 Commits

docs	Add files via upload	44 minutes ago
Asistente.java	Add files via upload	now
Casos.png	Add files via upload	41 minutes ago
Clases.png	Add files via upload	44 minutes ago
Diagrama sin titulo.drawio.png	Add files via upload	43 minutes ago
Evento.java	Add files via upload	now
EventoConcierto.java	Add files via upload	now
EventoConferencia.java	Add files via upload	now
EventoFactory.java	Add files via upload	now
GestorEventos.java	Add files via upload	now
IObservadorEvento.java	Add files via upload	now
README.md	Initial commit	2 hours ago

Readme
Activity
0 stars
0 watching
0 forks

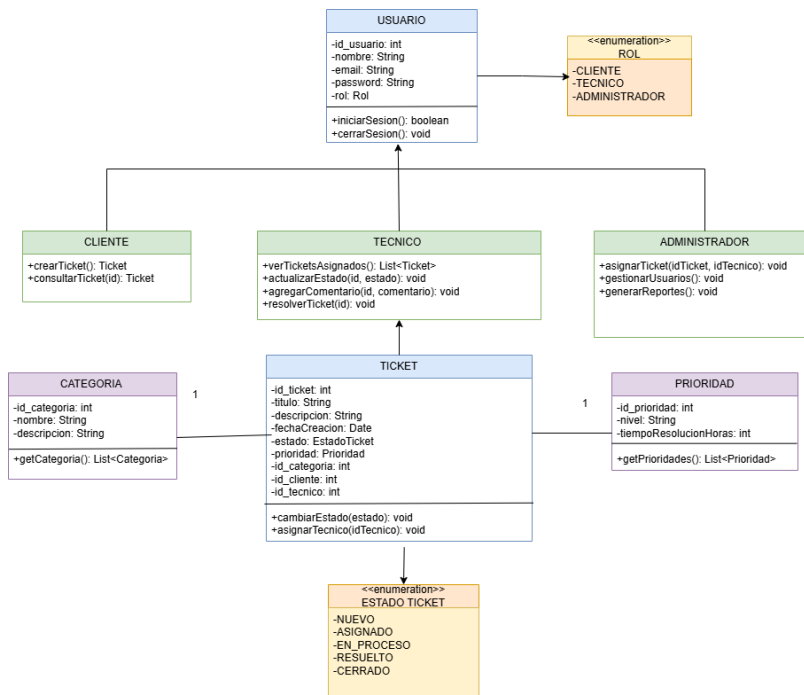
Releases
No releases published
[Create a new release](#)

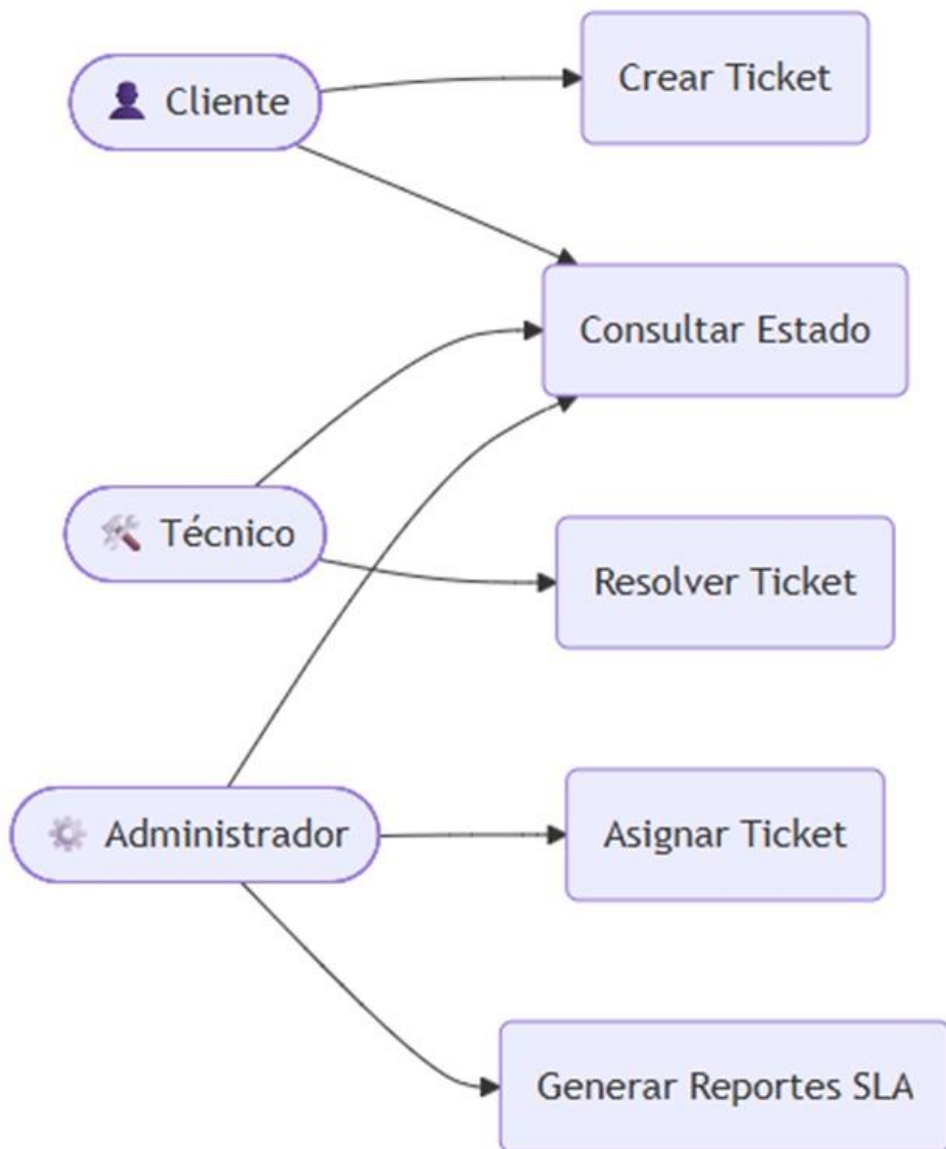
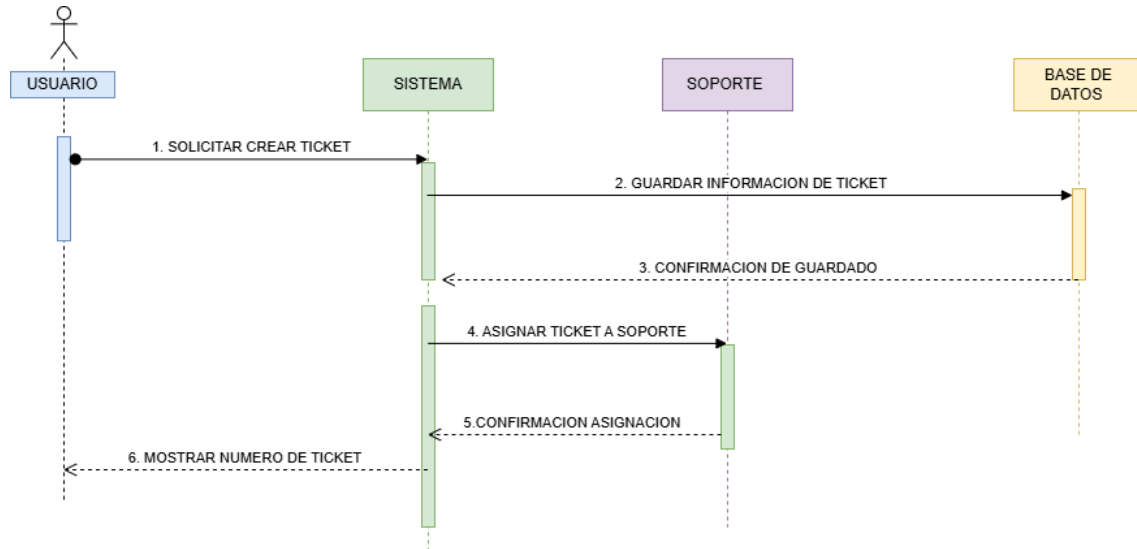
Packages
No packages published
[Publish your first package](#)

Contributors 1
avinces1033-rgb

31° ESP LAA 03:40 p. m. 15/05/2026

Evidencia 2 – Diagramas UML en Draw.io







12. Conclusión

El diseño del sistema Help Desk permitió aplicar conceptos fundamentales de ingeniería de software mediante diagramas UML, patrones de diseño y herramientas de control de versiones. La propuesta desarrollada facilita una administración organizada de incidencias técnicas y mejora la estructura lógica del sistema para futuras ampliaciones.

Bibliografía

- Pressman, R. S. Ingeniería del Software.
- Sommerville, I. Ingeniería de Software.
- Gamma, E. Patrones de Diseño.
- GitHub Documentation.
- Draw.io Documentation.



Evidencias

```
package sistema.gestor.eventos;

public class GestorEventos {
    public static void main(String[] args) {
        // Crear observadores
        Asistente maria = new Asistente("Maria");
        Asistente luis = new Asistente("Luis");

        // Crear gestor y registrar observadores
        GestorEventosManager gestor = new GestorEventosManager();
        gestor.agregarObservador(maria);
        gestor.agregarObservador(luis);

        // Crear evento usando Factory
        System.out.println("=== CREANDO EVENTO ===");
        Evento evento = EventoFactory.crearEvento("concierto", "Concierto de
música en vivo");
        evento.describirEvento();

        // Notificar a observadores
        gestor.notificarObservadores(evento);
    }
}

// Clase interna para manejar observadores
class GestorEventosManager {
    private java.util.List<IObservadorEvento> observadores = new
java.util.ArrayList<>();

    public void agregarObservador(IObservadorEvento obs) {
        observadores.add(obs);
    }

    public void notificarObservadores(Evento evento) {
        for (IObservadorEvento obs : observadores) {
            obs.actualizar(evento);
        }
    }
}
```

LINK DE GITHUB

<https://github.com/jmontano5898-jpg>

LINK DEL DRIVE

https://drive.google.com/drive/folders/1IH160lrZOOg7vud9D7FtRwz0_uUFwDm?usp=sharing